

Exp-24 Financial Data Analysis of Alphabet Inc. Using Line Charts

Aim

To write a Python program using Pandas and Matplotlib to draw line charts representing the Open, High, Low, and Close financial data of Alphabet Inc. from October 3, 2016 to October 7, 2016.

Procedure

- Import the required libraries, Pandas and Matplotlib.
- Create or load the `fdata.csv` file containing the financial data.
- Read the CSV file using `pandas.read_csv()`.
- Convert the Date column into the appropriate date format.
- Plot the Open, High, Low, and Close values against the dates.
- Add a title, axis labels, legend, and grid if required.
- Display the line chart using `plt.show()`.

CODE

```
24.py - C:/Users/subha/OneDrive/Desktop/Query processing/24.py (3.12.10)
File Edit Format Run Options Window Help
import pandas as pd
import matplotlib.pyplot as plt

# Read the financial data
data = pd.read_csv("fdata.csv")

# Convert Date column to datetime
data['Date'] = pd.to_datetime(data['Date'], format='%m-%d-%y')

# Draw line charts
plt.plot(data['Date'], data['Open'], label='Open')
plt.plot(data['Date'], data['High'], label='High')
plt.plot(data['Date'], data['Low'], label='Low')
plt.plot(data['Date'], data['Close'], label='Close')

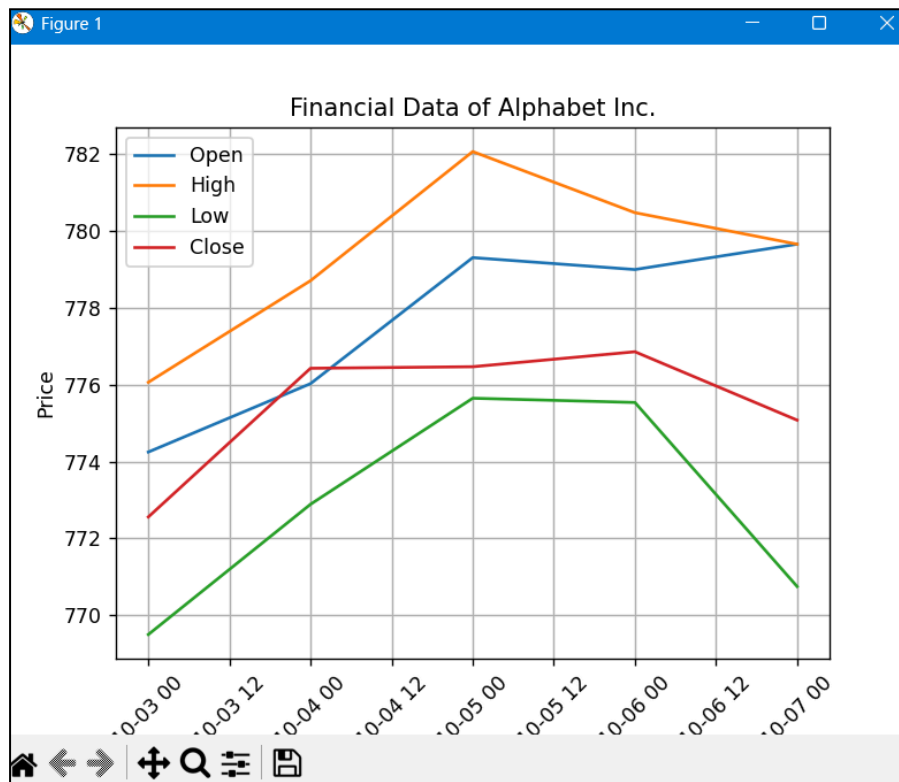
# Add labels and title
plt.xlabel('Date')
plt.ylabel('Price')
plt.title('Financial Data of Alphabet Inc.')

# Display legend
plt.legend()

# Format the graph
plt.xticks(rotation=45)
plt.grid(True)

# Display output
plt.show()
|
```

OUTPUT



RESULT

Thus, the Python program was successfully executed to draw line charts of Alphabet Inc.'s Open, High, Low, and Close financial data from October 3, 2016 to October 7, 2016.

Exp -25 Plotting Multiple Lines with Different Widths, Colors, and Legends Using Python

Aim

To write a Python program to plot two or more lines with different colors, line widths, and suitable legends using Matplotlib.

Procedure

- Import the `matplotlib.pyplot` library.
- Define the X-axis and Y-axis values for the lines.
- Plot the first line with a specified color and line width.
- Plot the second line with a different color and line width.
- Add suitable legends for both lines.
- Add X-axis and Y-axis labels and a title.
- Display the graph using `plt.show()`.

CODE

```
25.py - C:/Users/subha/OneDrive/Desktop/Query processing/25.py (3.12.10)
File Edit Format Run Options Window Help
import matplotlib.pyplot as plt

# X-axis values
x = [10, 20, 30]

# Y-axis values
line1 = [20, 40, 10]
line2 = [40, 10, 30]

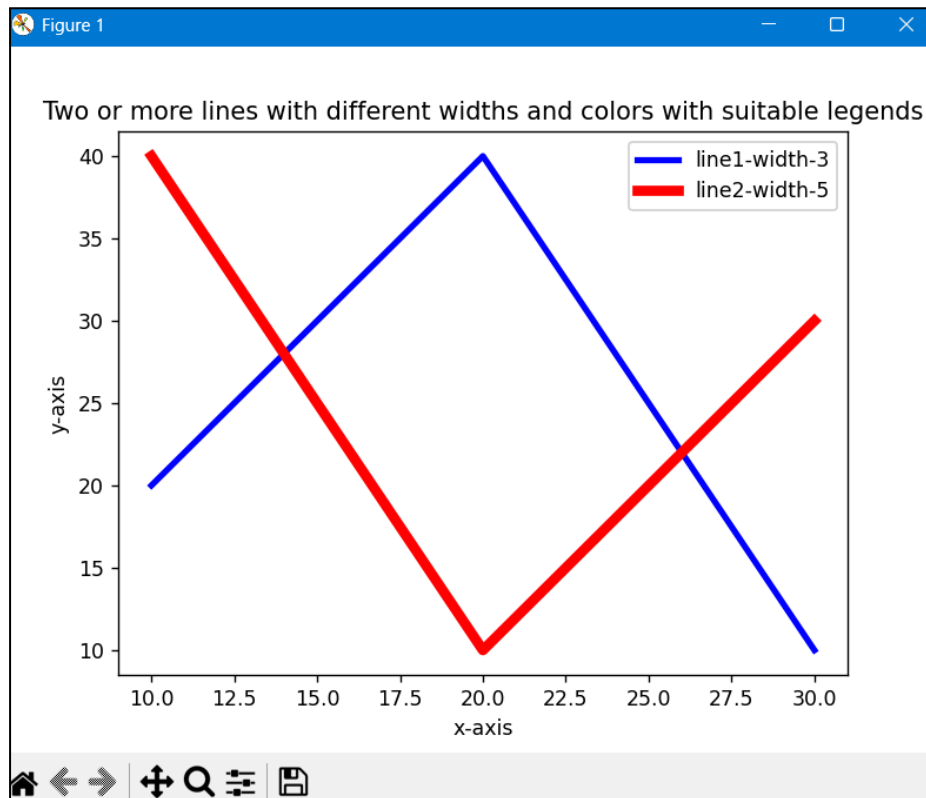
# Plot two lines with different colors and widths
plt.plot(x, line1, color='blue', linewidth=3, label='line1-width-3')
plt.plot(x, line2, color='red', linewidth=5, label='line2-width-5')

# Add labels and title
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.title('Two or more lines with different widths and colors with suitable legends')

# Display legend
plt.legend()

# Display the graph
plt.show()
```

OUTPUT



RESULT

Thus, the Python program was successfully executed to plot multiple lines with different colors, widths, and legends using Matplotlib.

Exp 26- Creating Multiple Plots Using Python

Aim

To write a Python program to create multiple plots in a single figure using Matplotlib.

Procedure

- Import the `matplotlib.pyplot` library.
- Create a figure using `plt.figure()`.
- Add multiple subplots using `plt.subplot()`.
- Plot different data in each subplot.
- Add suitable titles and labels.
- Display all the plots using `plt.show()`.

CODE

```
26.py - C:/Users/subha/OneDrive/Desktop/Query processing/26.py (3.12.10)
File Edit Format Run Options Window Help
import matplotlib.pyplot as plt

# Create a figure
plt.figure(figsize=(8, 6))

# First plot
plt.subplot(2, 1, 1)
plt.plot([1, 2, 3, 4], [10, 20, 15, 25])
plt.title("Line Plot")

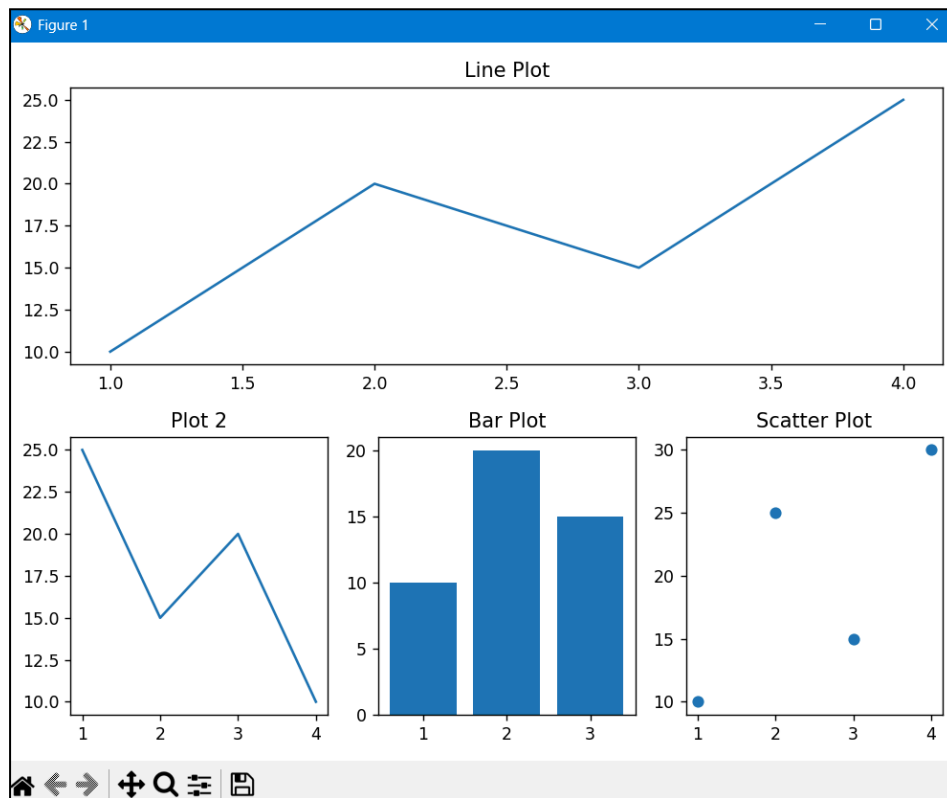
# Second plot
plt.subplot(2, 3, 4)
plt.plot([1, 2, 3, 4], [25, 15, 20, 10])
plt.title("Plot 2")

# Third plot
plt.subplot(2, 3, 5)
plt.bar([1, 2, 3], [10, 20, 15])
plt.title("Bar Plot")

# Fourth plot
plt.subplot(2, 3, 6)
plt.scatter([1, 2, 3, 4], [10, 25, 15, 30])
plt.title("Scatter Plot")

plt.tight_layout()
plt.show()
|
```

OUTPUT



RESULT

Thus, the Python program was successfully executed to create and display multiple plots in a single figure using Matplotlib.